

•



- [Operational Work](#)
- [Pulse Guide](#)
- Use Models

On this page

Use Models

This section points you to the Pulse sections that explain how you can build models and AutoML runs.

Build models📄

Pulse supports your Data Science loop by enabling you to quickly train your models and evaluate the results over several iterations. A model project aggregates several iterations of a model and provides tools for you to assess whether the features you designed are useful, well constructed, etc. At each iteration, you are able to understand how close you are to reaching your projects goals and what actions you took to reach those results. Building and evaluating your models with Pulse involves:

- [Sampling data](#)
- [Training a model](#)

- [Evaluating a model](#)
- [Tuning model parameters](#)
- [Comparing models](#)

Standard feature plan

Transaction Fraud for PSPs delivers an out-of-the-box standard feature plan. It contains a set of features identified by Feedzai from industry knowledge and experimentations with historical data.

The feature model can be used as a placeholder to rapidly create a ML model into the "TFP Scoring Model" workflow element, which serves as a placeholder for your [ML model](#).

Feature Plan

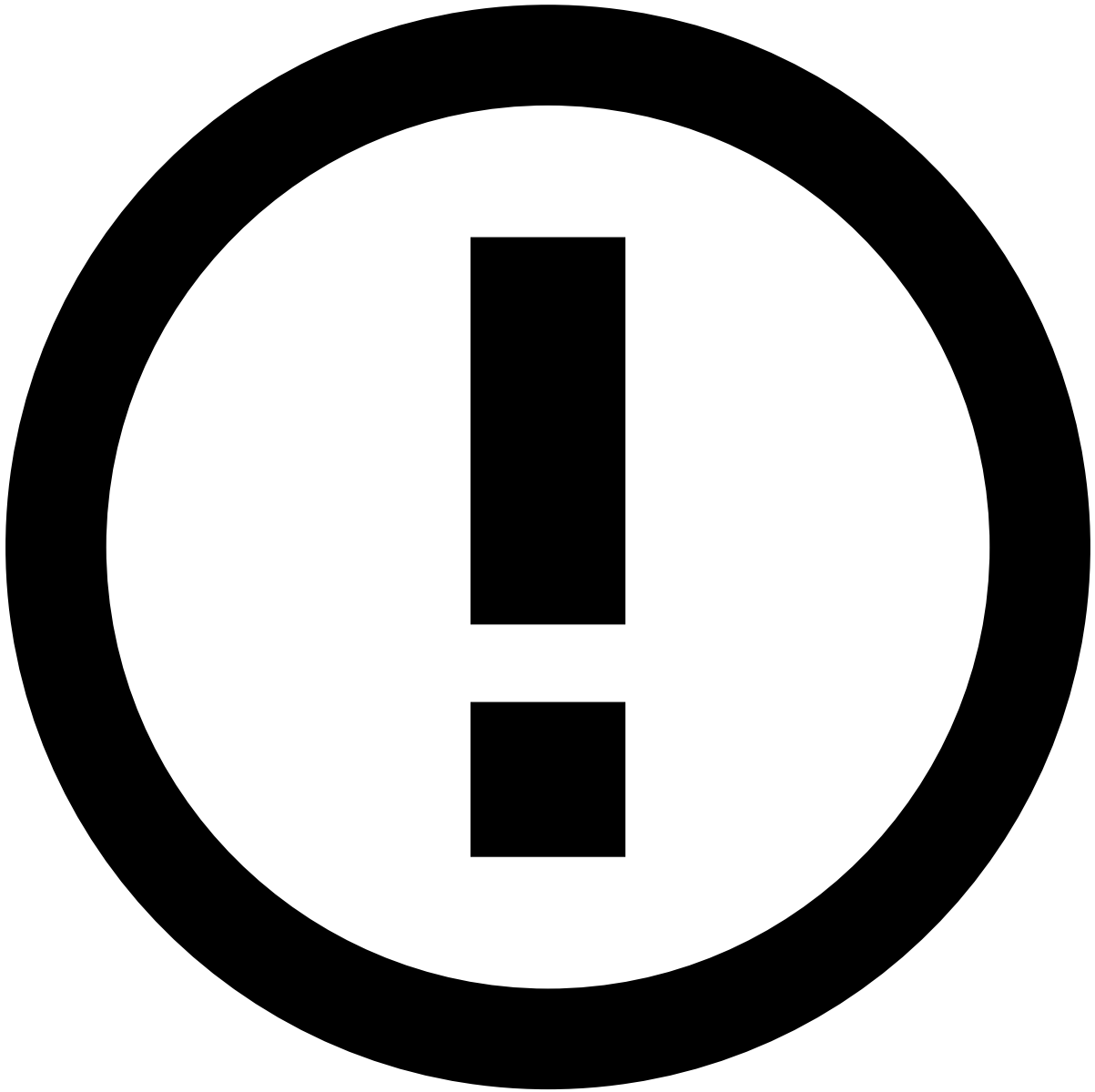
- **Name:** PSP Model Features Plan.
- **Output schema:** Features Plan Schema (Payments Workflow Schema + fields calculated on the PSP Model Features Plan). Only a model should use this schema.
- **Position on the workflow:** After Model Features Plan and before all the rules.

Model

- **Name:** TFP Scoring Model.
- **Output schema:** Uses a simple schema to minimize TFP Scoring Model's processing impact. Note that, to start using a model, you should use the Features Plan Schema instead.
- **Placeholder model:** Always scores events with 0.
- **Position on the workflow:** Before the decision rules block.

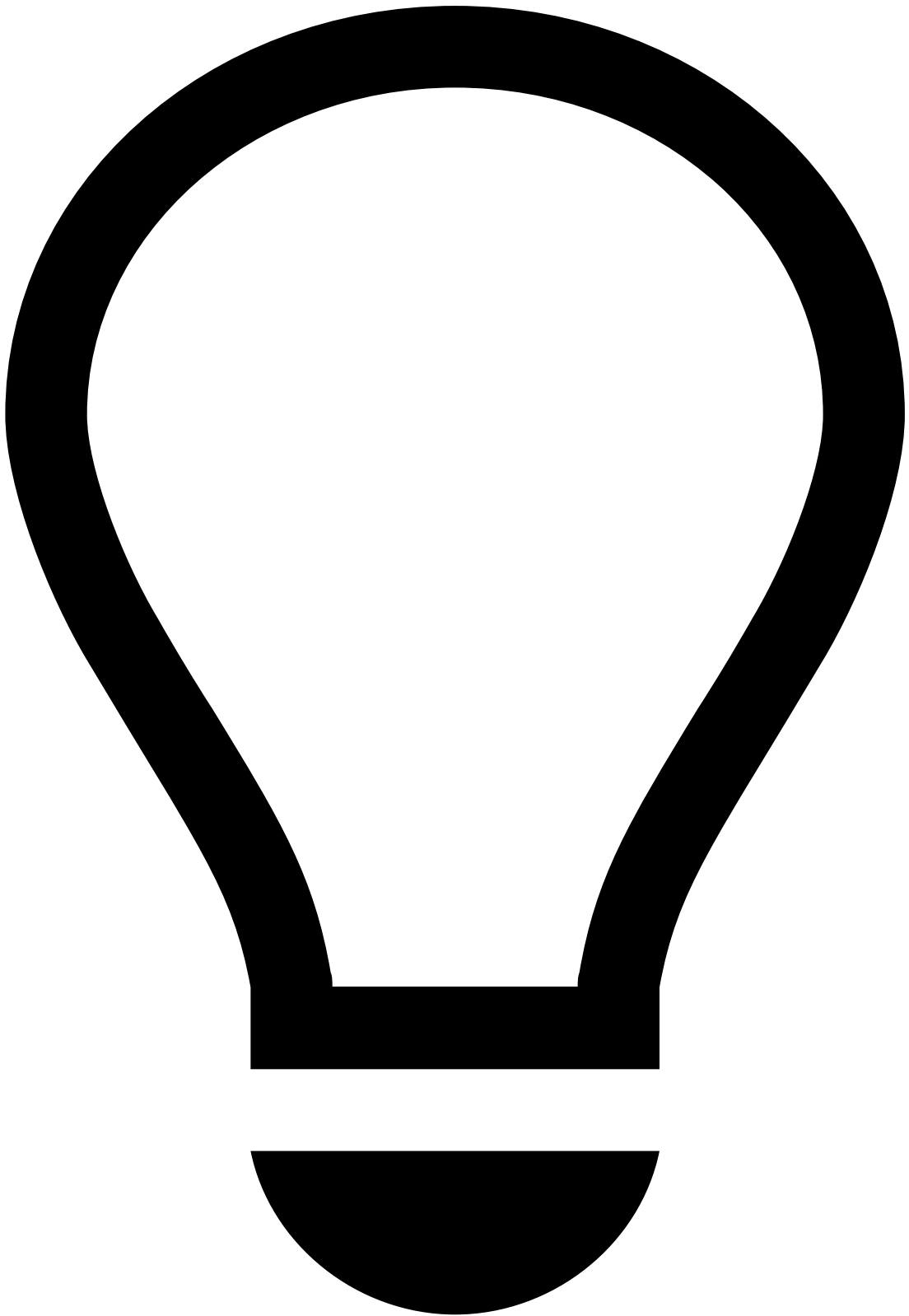
Features are grouped into operations / data transformations according to their type, e.g. map operations are grouped into "Feature group 1".

The following table includes the standard features in the PSP Model Features Plan.



info

To see the features in the PSP Rules Metrics Plan, check the tables in the [Rule Metrics](#) page.



tip

Use the table's **horizontal scroll bar** to see all its columns.

Feature group 1	Feature group 2	Profiles	Feature group 3
dayOfMonth_event_occurred_at	cos_dayOfWeek_event_occurred_at	merchant_id_avg_amount_unified_currency_1_hour	probability_log_am
dayOfWeek_event_occurred_at	cos_hour_event_occurred_at	merchant_id_avg_amount_unified_currency_1_day	probability_log_am

emailName_billing_email		merchant_id_avg_log_amount_unified_currency_1_hour	probability_log_ame
norm_billing_email		merchant_id_avg_log_amount_unified_currency_1_day	ratio_merchant_id_
hour_event_occurred_at		merchant_id_countDistinct_customer_id_1_hour	ratio_merchant_id_
log_amount_unified_currency		merchant_id_countDistinct_merchant_mcc_1_hour	ratio_merchant_id_
month_event_occurred_at		merchant_id_countDistinct_payment_card_id_1_hour	ratio_merchant_id_
		merchant_id_countDistinct_payment_card_bin_1_hour	ratio_merchant_id_
		merchant_id_countDistinct_billing_email_1_hour	ratio_merchant_id_
		merchant_id_countDistinct_customer_id_1_day	ratio_merchant_id_
		merchant_id_countDistinct_merchant_mcc_1_day	ratio_merchant_id_
		merchant_id_countDistinct_payment_card_id_1_day	ratio_merchant_id_
		merchant_id_countDistinct_payment_card_bin_1_day	ratio_merchant_id_
		merchant_id_countDistinct_billing_email_1_day	ratio_merchant_id_
		merchant_id_count_1_hour	ratio_merchant_id_
		merchant_id_count_1_day	ratio_merchant_id_
		merchant_id_max_amount_unified_currency_1_hour	ratio_merchant_id_
		merchant_id_max_amount_unified_currency_1_day	ratio_merchant_id_
		merchant_id_stdDev_amount_unified_currency_1_hour	ratio_merchant_id_
		merchant_id_stdDev_amount_unified_currency_1_day	ratio_merchant_id_
		merchant_id_stdDev_log_amount_unified_currency_1_hour	

	merchant_id_stdDev_log_amount_unified_currency_1_day
	merchant_id_sum_amount_unified_currency_1_hour
	merchant_id_sum_amount_unified_currency_1_day
	merchant_id_timeSincePrev_event_occurred_at_1_hour
	merchant_id_timeSincePrev_event_occurred_at_1_day
	merchant_id_vel_1_hour
	merchant_id_vel_1_day
	merchant_id_avg_amount_unified_currency_1_month
	merchant_id_avg_log_amount_unified_currency_1_month
	merchant_id_count_1_month
	merchant_id_max_amount_unified_currency_1_month
	merchant_id_stdDev_amount_unified_currency_1_month
	merchant_id_stdDev_log_amount_unified_currency_1_month
	merchant_id_sum_amount_unified_currency_1_month
	merchant_id_timeSincePrev_event_occurred_at_1_month
	merchant_id_vel_1_month

The following sections help you build a model using the standard feature plan, and explain how models can improve rule performance.

Use the standard feature plan to build a model🔗🔗

Use the out-of-the-box standard feature plan, i.e. **Model Features Plan**, to build a model and then run the Model Features Plan.

Import a data source🔗🔗

To train an algorithm and build a model, you need a set of events described through fields and labels. This dataset is used as input, over which the algorithm will iterate until it learns and recognizes patterns. These patterns allow to establish relationships between events's features and the labels that the model intends to predict. The labels are typically referred to as classes.

The Transaction Fraud for PSPs use case was designed to handle events labeled as legitimate or fraudulent. In more specific terms, the dataset is expected to have a field `fraud` that indicates whether or not an event is fraudulent (`fraud == False` or `fraud == True`).

To import your dataset into Pulse, you must follow the procedure described in the [Importing your Dataset into Pulse](#) guide.

1. When importing your dataset, choose the option **Import from an Existing Schema** on the **Data Source Characterization** tab:
 1. A pop up window will allow you to **Select the Pre Omnichannel Metrics Plan Schema** and confirm. Pulse will indicate if your dataset is compatible with the solution's input schema:
 1. Save and create the data source.

If your raw data schema does not match the input data schema expected by the solution (i.e. **Pre Omnichannel Metrics Plan Schema**), you can modify your raw data. The [Validating the Data Schema](#) guide helps you validate and convert your data.

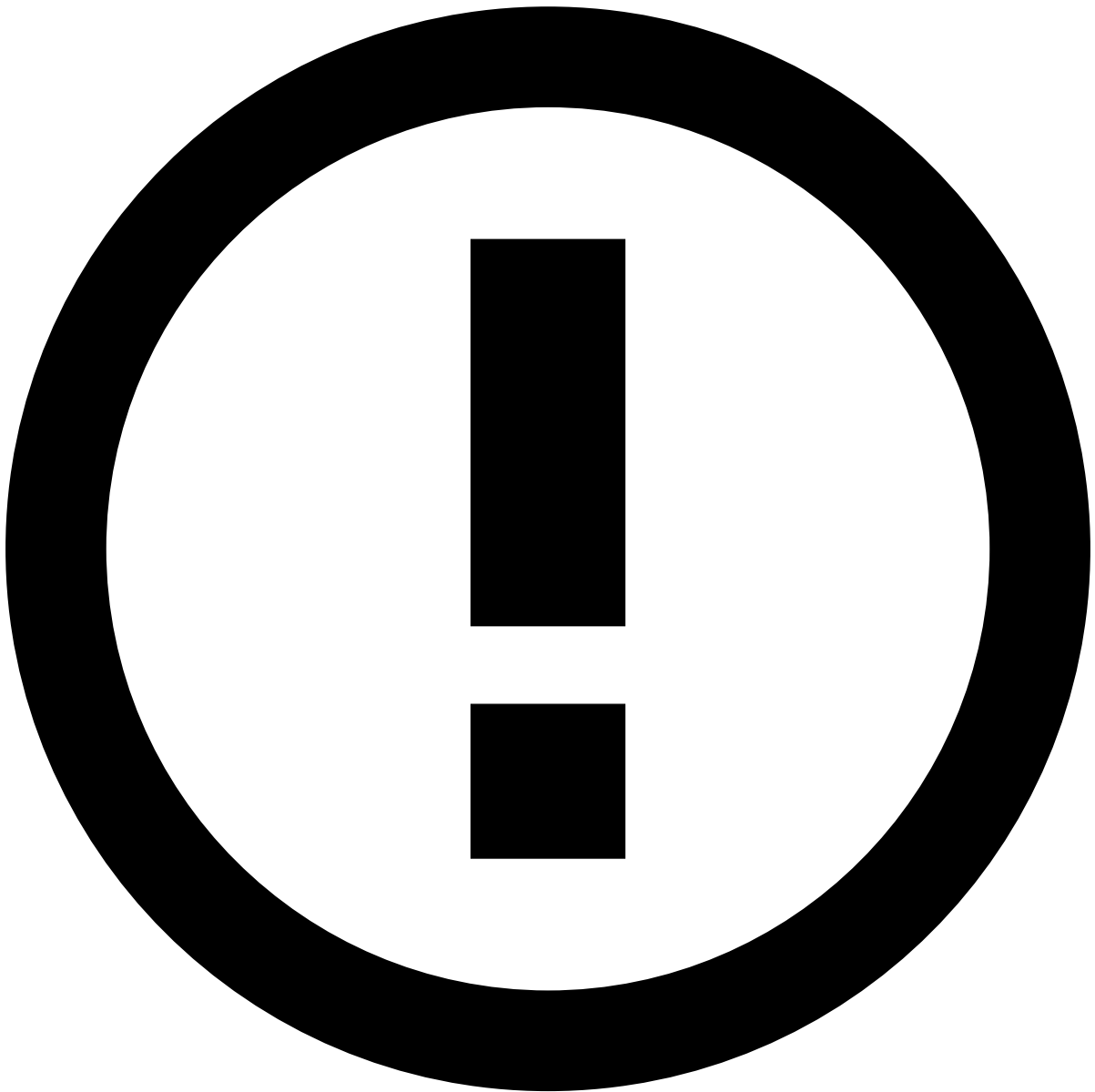
You can learn more in [Cleaning up Data](#).

Run the standard feature plan

Typically, you want to train your machine learning model on a dataset containing derived fields and historical profiles, or metrics. Historical profiles describe how a particular entity behaves and can be used to identify deviations or anomalies. Certain changes in the behavior may help you detect financial crime. For example, consider the scenario where a customer who only uses their credit card to pay small amounts on monthly streaming services, suddenly starts making daily purchases at much higher amounts than usual.

You can achieve this by building up your feature plan and generating a feature dataset. The Model Features Plan is designed to compute pre-defined derived fields and metrics. In this plan, an input data source is processed, metrics are computed on top of it, and the resulting data source is persisted.

To compute these derived fields and metrics for your dataset, you should execute the Model Features Plan using your dataset as the Plan input data source. The process of running the plan to generate the enhanced data source is described in the [Generating the Features Dataset](#) guide.



info

For better model performance, combine the features in the PSP Model Features Plan with the PSP Rules Plan.

Read the [Build and evaluate a model](#) section to learn how to run the standard feature plan in order to build your ML model section. The same process should be followed for the PSP Rules Plan in case you want to use its features in the model.

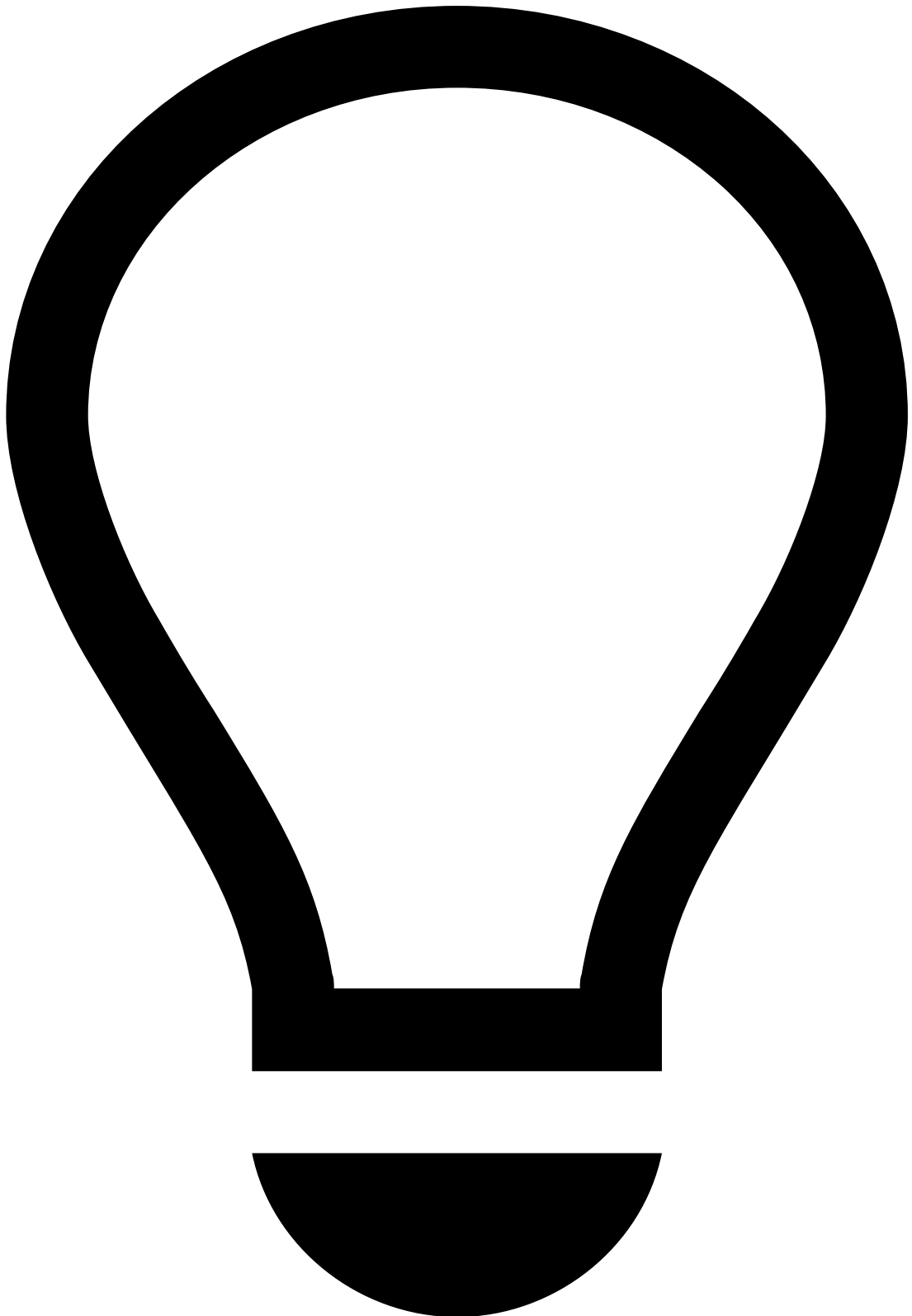
Data Science Platform considerations

Within the data science platform there are some considerations to take into account. The following table highlights scalability concerns that are relevant when executing plans.

	Context	Implication
Spark Jobs & data volume	Metrics done in the Data Science Platform make use of Spark to distribute the workload across a Hadoop cluster.	

	Distributing metrics through a Spark job means that data will be moved around in the cluster in large quantities. Due to the nature of profiles, each entity's value (e.g. each card or account) will be processed in the same Spark machine (or executor).	Even though a metric may only need 2 or 3 fields to compute, Spark will still move the records with all their columns to the executors.
Spark Jobs & skewness	Very high skews in grouping entities (e.g. 10% of entities represent 90% of the total records), can make Spark Tuning more complex.	Avoid including highly skewed grouping entities in your metrics. If they cannot be avoided, consider the following: • Randomize special entities (e.g. government-related beneficiaries, gmail, etcâ).) • Salt the largest entities: added cost of joining the features.

To understand more about the implications of distributed Spark Jobs within the Data Science Platform, check the relevant [data partitioning documentation](#).



tip

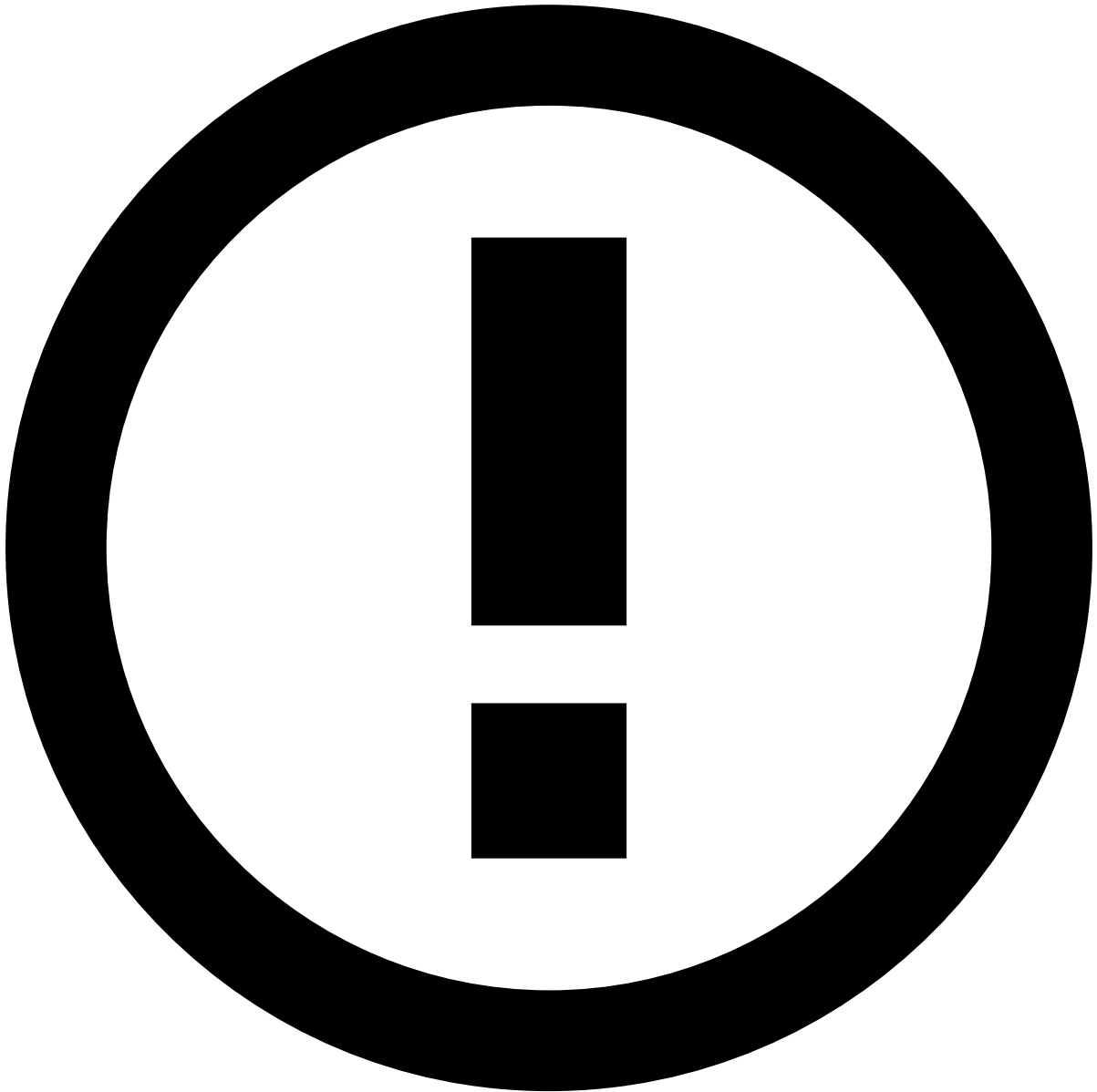
Pulse allows you to configure data partitioning using flexible UI options to ensure that Spark performs data re-organization as homogenous and well distributed as possible. Data partitioning is a key step for processing distributed jobs in a Spark cluster spawned by plan executions to enable Pulse to efficiently handle large volumes of data. The data-aware optimized partitioning is enabled by default. You may see this functionality in the **Advanced Options** tab when configuring a Plan Execution (see image below). To learn more about it, read the [Data Partitioning for the Metrics Operator](#) section.

Troubleshooting

1. Missing fields: The schema of the dataset to be evaluated may not correspond the schema expected by the Model Features Plan (Pre Omnichannel Metrics Plan Schema). Therefore, pay attention to field names, type, and order when importing a dataset.

If you are unable to include a particular field present in the Pre Omnichannel Metrics Plan Schema in the to-be-evaluated dataset, observe the following scenarios and their consequences:

- The missing field is not used by the plan
 - **Impact:** This scenario has little impact on the model creation pipeline, as it only requires the user to change the input schema used by the Plan.
 - **Workaround:** Duplicate the Model Features Plan and just modify the schema used in the Plan Settings, changing the Plan input schema to the one defined by your dataset. Don't forget to set the timestamp field to `event_occurred_at`. No Operator in the Plan should be modified.
- The missing field is used by the plan
 - **Impact:** This scenario may have a higher impact on the model creation pipeline, as it requires you to modify the Plan Operators. Since fields and metrics derived from missing fields cannot be created, they must be removed so that the plan can run. Note that by removing derived fields and metrics, the model performance can be affected.
 - **Workaround:** Duplicate the Model Features Plan to avoid losing the original Plan. In the duplicate plan, you must identify those to-be-created or -updated fields that depend on a given missing field, and consequently, cannot be calculated. As it will not be possible to generate these new fields or metrics, they should be removed from the Plan Operators. Modify the schema used in the Plan Settings, changing the Plan input schema to the one defined by your dataset. Don't forget to set the timestamp field to `event_occurred_at`.



info

When the `customer_email` field is not available in payment events, the PSP Rules Plan will coalesce the missing `customer_email` field to the `billing_email` field.

1. Grouping entities with several missing values: Suppose you want to compute profiles grouped by the `device_ip` field but, in your dataset, about half of the transactions don't have a device IP address. You start by adding a filter to the metric window, e.g. `device_ip != null`, but this is not enough.

When computing the profile, all the transactions will still be loaded and distributed to the executors before the filter is applied. There will be a very large partition with all the missing IPs, which will likely crash the Spark job.

1. Replace that missing value with something else, such as the transaction ID (`event_external_id`) or the event timestamp (`event_occurred_at`).
2. Create a new field `device_ip_salted` in a Map Operator before the Metrics Operator.
3. Assign it a value similar to `device_ip ?? event_external_id`.
4. Update the metric window filter to `device_ip_salted != event_external_id`. This will still yield the same end result, but without generating a massive partition in the Spark job.

Edit Map Operator

Learn more in:

- [Creating Derived Features](#)
- [Creating Historical Profiles](#)
- [Best Practices for Plans](#)

Build and evaluate a model

As the output of the Model Features Plan, you will have a new dataset with the original features and the additional features created through the Plan. You should use this new dataset to train a model.

If the dataset you are going to use to build your model does not have all the data labeled, you can adopt one of the following strategies:

- Remove the unlabeled events from the dataset.

OR

- Coalesce the unlabeled events to a default value. For example, you may assume that transactions that were not deliberately marked as fraudulent are legitimate, and therefore, should have the fraud `label = False`.

A possible workaround for the missing labels is to run a Pulse Plan using a Map Operator to create a new label field or update an existing label field, as the following image shows:

Typically, data scientists use different data splits to train, validate, and optionally, test a model. There shouldn't be any overlap between the training, validation, and test sets. As a general machine learning guideline, these splits are 80/20/20, i.e., 80% of your data for training, 20% for validation, and 20% for testing (hold-out data set). Here, the split is enforced to be defined timewise, because the transformations are time-sensitive.

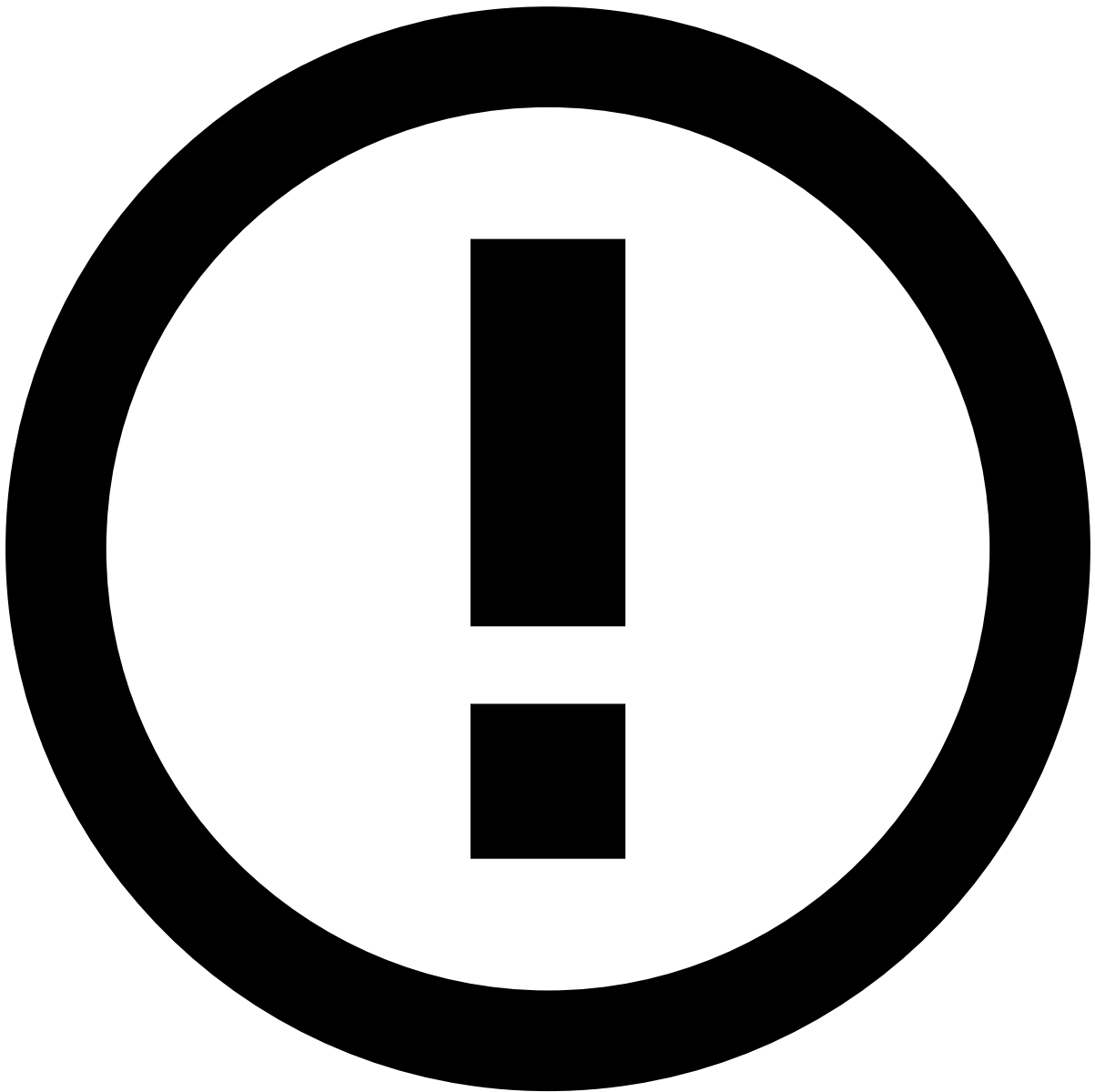
The training, validation, and test splits can be generated through JupyterLab or by creating and running a new Pulse Plan. If you choose to partition the data using a Pulse Plan, you will have to create a Filter Operator for each split that you want to generate.

As a usage example, imagine that you want to create three splits, e.g. one for training, one for validation, and one for test:

The condition to be observed for the creation of these splits is a temporal criterion based on the timestamp value of the `event_occurred_at` field.

This means that, for the three filters, you will have the following conditions:

- Training Split: `event_occurred_at < timestamp1`
- Validation Split: `event_occurred_at >= timestamp1 and event_occurred_at < timestamp2`
- Test Split: `event_occurred_at >= timestamp2`



info

The following condition must be observed: `timestamp1 < timestamp2` . If the dataset is extremely unbalanced, the same Pulse Plan can also be used to sample the most frequent class.

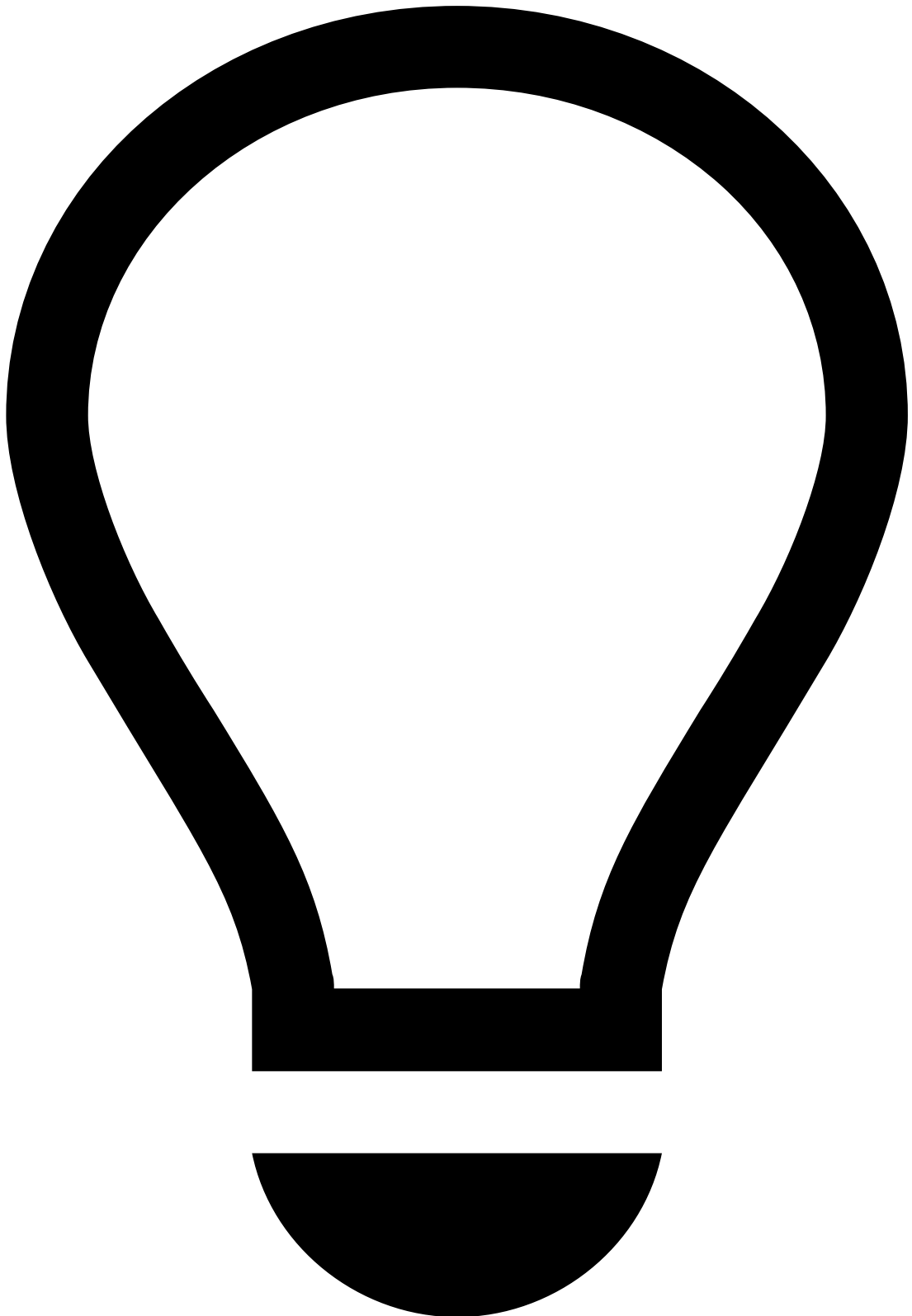
1. Separate the events belonging to each class using a Filter Operator.
2. Use a Sample Operator to reduce the number of events in the most frequent class.
3. The sampled majority class and the instances belonging to the minority class must pass through a Union Operator to compose the new dataset.

After running the splitting Plan in Pulse, the new dataset splits will be listed in the Data Sources section on the **Data Science** tab. The new dataset splits will have the same schemas and can be used in the usual training and evaluation model pipeline.

1. Use the training split to train your model. You may also use the validation split to [validate your model](#). In the **Validation & Reporting** tab, select the Validation **Validation Data Source** type and choose the validation dataset split that you have just created.

Learn more in [Evaluating a Model](#).

1. Use the test split to evaluate your model.



tip

When training an algorithm, the features in the dataset are expected to have a different impact on the model performance. You may find features that contribute to the improvement of the model's performance and others that just add noise to the model. It is, therefore, recommended that you run an analysis on your dataset to [examine the statistics for each field](#). For example, categorical fields with only one category, e.g. empty value, should not be considered when training your model.

You can also [compute the Feature Importance](#) and remove the features that had a negative impact on the previous model iteration.

Learn more in:

- [Tuning Model Parameters](#)
- [Comparing Models](#)

How models improve rule performance🔗

What is the model score?🔗

A machine learning model is used to label new (unseen) events. In a classification scenario, where a binary model is used to predict fraud events, the model assigns a value (score) representing its confidence that a given event belongs to one of the classes (fraud and legit). Typically, a fraudulent event is characterized as having the field `fraud == True`, while legitimate events are represented by events having the field `fraud == False`.

The score generated by Pulse models corresponds to a probability of an event belonging to the positive class (`fraud == True`), thus ranging from 0 to 1. However, to make the interpretation easier, Pulse returns the score of a model multiplied by 1000. Thus, the score assigned by the model to the positive class will always vary between 0 and 1000. The greater the confidence of the model in labeling an event as `fraud`, the higher the score returned.

Where to position the model in the workflow?🔗

The machine learning model trained on the raw and derived features should be inserted into the **Payments Workflow** using a Scoring operator. Use the TFP Scoring Model operator that is included in the out-of-the-box workflow (see image below). This way, the model can use the derived fields and metrics that are generated by this plan.

When configuring your model, select the Model Project and the Model dropdown fields. Ensure that the target outcome of the model is compatible with the selected outcome. The following image presents an example of a model configuration:

How is the model score used in the workflow?🔗

In the **Transaction Fraud for PSPs Workflow**, the model score produced will be evaluated by [decision rules](#). When evaluating the model score, the other rule blocks will take precedence over the [model's decision](#). Therefore, if the PosNeg rule block triggers an `approve` for a payment event, the workflow will have the final outcome `approve`. Also, if any rules block triggers a decline for a payment event, the workflow will have the final outcome `decline`, regardless of the model result.